LAPORAN PRATIKUM
STRUKTUR DATA

Disusun Oleh:

Rifqi Aditya

2511533002

Dosen Pengampu :

Dr. Wahyudi, S.T, M.T

Asisten Praktikum :

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum mata Struktur Data tentang *ADT*.

Laporan ini disusun untuk memenuhi salah satu tugas praktikum sekaligus sebagai sarana pembelajaran dalam memahami dasar-dasar pemrograman menggunakan bahasa Java. Melalui praktikum ini, penulis dapat mempelajari penggunaan ADT.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses penyusunan laporan ini.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca, khususnya dalam memperdalam pemahaman tentang pemrograman dasar dengan Java.

Padang, 2 April 2025

Penulis

Daftar Pustaka

BAB I PENDAHULUAN 4
1.1 Latar Belakang 4
1.2 Tujuan 4
1.3 Manfaat 4
BAB II PEMBAHASAN 5
2.1. Uraian Kode Program 5
2.2 Output 11
BAB III TUGAS 12
3.1 Uraian Kode Program 12
3.2 Output 14
BAB IV KESMPULAN 15

BAB I
PENDAHULUAN

1.1 Latar Belakang

ADT merupakan suatu konsep dalam pemrograman yang menekankan pada pemisahan antara definisi data dan implementasinya. Dengan menggunakan ADT, pengguna hanya perlu mengetahui operasi-operasi yang dapat dilakukan terhadap suatu data tanpa harus memahami bagaimana data tersebut diimplementasikan secara internal. Hal ini berkaitan erat dengan konsep Abstraksi, yang bertujuan untuk menyederhanakan sistem dengan menyembunyikan detail yang tidak diperlukan.

Penerapan ADT sangat penting dalam meningkatkan kualitas perangkat lunak, karena memungkinkan pengembangan program yang lebih modular, mudah dipelihara, dan fleksibel. Oleh karena itu, pemahaman mengenai ADT menjadi dasar yang penting dalam pembelajaran struktur data dan algoritma.

1.2 Tujuan

1. mampu memahami prinsip fungsional dan implementasi suatu struktur data.
2. mampu merancang dan membuat tipe data baru menggunakan struktur.
3. Melatih kemampuan dalam mengorganisasikan kode program ke dalam modul-modul yang terpisah

1.3 Manfaat

1. Memberikan pemahaman mendalam mengenai cara membangun arsitektur program yang terstruktur dan sistematis.
2. enjadi lebih terampil dalam memecahkan masalah kompleks dengan cara membaginya menjadi komponen-komponen data
3. Menjadi landasan fundamental bagi mahasiswa sebelum mempelajari konsep yang lebih kompleks seperti *Linked List*, *Stack*, *Queue*, dan *Tree*.

BAB II
PEMBAHASAN

2.1. Uraian Kode Program

1.Class Jam

```java
package Pekan1_2511533002;

public class Jam_2511533002 {
    private int hh; //0..23
    private int mm; //0..59
    private int ss; //0..59

    //Validator
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
    }

    //Konstruktor/Kreator
    public Jam_2511533002(int h, int m, int s) {
        this.hh = h; this.mm = m; this.ss = s;
    }

    //Selektor
    public int getHour() { return hh; }
    public int getMinute() { return mm; }
    public int getSecond() { return ss; }

    //Mutator
    public void setHour (int h) {this.hh = h;}
    public void setMinute (int m) {this.mm = m;}
    public void setSecond (int s) {this.ss = s;}

    //Konversi
    public int toSeconds() { return hh * 3600 + mm + 60 + ss; }
    public static Jam_2511533002 fromSecond (int total) {
        if (total < 0) throw new IllegalArgumentException("detik negatif");
        total %= 24 * 3600; //Wrap 24 jam
        int h = total / 3600; total %= 36000;
        int m = total / 60; int s = total % 60;
        return new Jam_2511533002(h, m, s);
    }

```

```java

    //Relasional
    public int compareTo (Jam_2511533002 other) { return Integer.compare(this.toSeconds(), other.toSeconds()); }
    public boolean equals(Object o) {
        if (!(o instanceof Jam_2511533002 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.ss;
    }
    public int  hashCode() {return java.util.Objects.hash(hh, mm, ss); }

    //Aritmatika
    public Jam_2511533002 plus(Jam_2511533002 other) {return fromSecond(this.toSeconds() + other.toSeconds());}
    public Jam_2511533002 minus(Jam_2511533002 other) {return fromSecond(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600));}
    public Jam_2511533002 nextSecond() {return fromSecond(this.toSeconds() + 1);}
    public Jam_2511533002 nextNSeconds(int n) {return fromSecond(this.toSeconds() + Math.max(0, n));}
    public Jam_2511533002 prevSecond() {return fromSecond(Math.floorMod(this.toSeconds() - 1, 24 * 3600));}
    public Jam_2511533002 prevNSeconds (int n) {return fromSecond(Math.floorMod(this.toSeconds() - Math.max(0, n), 24 * 3600));}

//Durasi (detik)
public static int durasiDetik(Jam_2511533002 jaw, Jam_2511533002 jakh) {return jakh.toSeconds() - jaw.toSeconds();}
//Overriding ke string
public String toString() {return String.format("%02d:%02d:%02d", hh, mm, ss);}}
```

1. Validator

- public static boolean isValid(int h, int m, int s) { : Fungsi untuk mengecek apakah angka jam, menit, dan detik yang dimasukkan valid atau tidak.
- return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59); : Mengembalikan nilai true jika jam 0-23, menit 0-59, dan detik 0-59. Jika tidak, maka nilai di kembalikan atau false.

2. Konstruktor
   - this.hh = h; this.mm = m; this.ss = s; : Memasukkan nilai parameter h, m, s ke dalam atribut objek.
3. Getter
   - public int getHour() { return hh; } : Mengambil nilai jam.
   - public int getMinute() { return mm; } : Mengambil nilai menit.
   - public int getSecond() { return ss; } : Mengambil nilai detik.
4. Setter
   - public void setHour (int h) {this.hh = h;} : Mengubah nilai jam.
   - public void setMinute (int m) {this.mm = m;} : Mengubah nilai menit.
   - public void setSecond (int s) {this.ss = s;} : Mengubah nilai detik.
5. Konversi
   - public int toSeconds() { return hh * 3600 + mm + 60 + ss; } : Mengonversi waktu menjadi detik.
   - public static Jam_2511533002 fromSecond (int total) { : Fungsi untuk membuat Jam baru dari total detik.
   - if (total < 0) throw new IllegalArgumentException("detik negatif"); : Jika total detik negatif, program akan berhenti dan memunculkan error.
   - total %= 24 * 3600; : Melakukan pembatasan. Jika total detik lebih dari satu hari maka akan kembali ke 0.
   - int h = total / 3600; total %= 36000; : Menghitung jam.
   - int m = total / 60; int s = total % 60; : Menghitung sisa menit dan sisa detik.
   - return new Jam_2511533002(h, m, s); : Mengembalikan objek Jam yang sudah dikonversi.
6. Relasional
   - public int compareTo (Jam_2511533002 other) { return Integer.compare(this.toSeconds(), other.toSeconds()); } : Membandingkan dua objek jam.

- public boolean equals(Object o) { : Mengecek apakah dua objek jam dianggap identik.
- if (!(o instanceof Jam_2511533002 j)) return false; : Jika objek yang dibandingkan bukan tipe Jam, maka di anggap tidak sama.
- return hh == j.hh && mm == j.mm && ss == j.ss; : Dianggap sama jika jam, menit, dan detiknya sama.

7. public int hashCode() {return java.util.Objects.hash(hh, mm, ss); } : Membuat kode unik untuk objek tersebut

8. Aritmatika

   - public Jam_2511533002 plus(Jam_2511533002 other) {return fromSecond(this.toSeconds() + other.toSeconds());} : Menambahkan dua waktu.
   - public Jam_2511533002 minus(Jam_2511533002 other) {return fromSecond(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600));} : Mengurangi dua waktu.
   - public Jam_2511533002 nextSecond() {return fromSecond(this.toSeconds() + 1);} : Menambah waktu sebanyak 1 detik.
   - public Jam_2511533002 nextNSeconds(int n) {return fromSecond(this.toSeconds() + Math.max(0, n));} : Menambah waktu sebanyak n detik.
   - public Jam_2511533002 prevSecond() {return fromSecond(Math.floorMod(this.toSeconds() - 1, 24 * 3600));} : Mundur 1 detik.
   - public Jam_2511533002 prevNSeconds (int n) {return fromSecond(Math.floorMod(this.toSeconds() - Math.max(0, n), 24 * 3600));} : Mundur sebanyak n detik.

9. Durasi

   - public static int durasiDetik(Jam_2511533002 jaw, Jam_2511533002 jakh) {return jakh.toSeconds() - jaw.toSeconds();} : Menghitung selisih detik antara jam awal dan jam akhir.
   - public String toString() {return String.format("%02d:%02d:%02d", hh, mm, ss);}: Mengubah objek jam menjadi teks string dengan format dua digit (contoh: 08:05:09).

2.Class Driver

```java
package Pekan1_2511533002;

public class JamDriver_2511533002 {

    public static void main(String[] args) {
        Jam_2511533002 a = new Jam_2511533002(23, 59, 50);
        Jam_2511533002 b = new Jam_2511533002(0, 0, 15);
        System.out.println("a        = "+a);
        System.out.println("b        = "+b);
        System.out.println("a+b      ="+a.plus(b));
        System.out.println("next 20s = "+a.nextNSeconds(20));
        System.out.println("durasi (a,b)       = "+Jam_2511533002.durasiDetik(a, b));

    }

}
```

- Jam_2511533002 a = new Jam_2511533002(23, 59, 50); : Membuat objek jam a dengan waktu hampir tengah malam, yaitu pukul 23:59:50.
- Jam_2511533002 b = new Jam_2511533002(0, 0, 15);: Membuat objek jam b dengan waktu dini hari, yaitu pukul 00:00:15.
- System.out.println("a = "+a); : Mencetak objek a.
- System.out.println("b = "+b);: Mencetak objek b.
- System.out.println("a+b ="+a.plus(b)); : Memanggil method plus. Ini menjumlahkan total detik a dan b. Jika hasilnya melebihi 24 jam, maka akan kembali ke hari awal karena adanya operator % di method fromSecond.
- System.out.println("next 20s = "+a.nextNSeconds(20)); : Mengambil waktu a lalu ditambah 20 detik,
- System.out.println("durasi (a,b) = "+Jam_2511533002.durasiDetik(a, b)); : Memanggil method durasiDetik untuk menghitung selisih detik antara jam a dan jam b.

3.Class Driver2

```java
package Pekan1_2511533002;

import java.util.Scanner;

public class JamDriver2_2511533002 {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("== Program Driver Objek Jam ==");

        //input jam pertama
        System.out.println("\n--- Input Jam 1 ---");
        Jam_2511533002 j1 = buatJamDariInput(input);

        //input jam kedua
        System.out.println("\n--- Input Jam 2 ---");
        Jam_2511533002 j2 = buatJamDariInput(input);

        //menampilkan data
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (string)          = "+j1.toString());
        System.out.println("Jam 2 (string)          = "+j2.toString());
        System.out.println("Jam 1 dalam detik       = "+j1.toSeconds());
        System.out.println("Jam 2 dalam detik       = "+j2.toSeconds());

        //operasi rasional (perbandingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println("Status                      : Jam 1 lebih lambat (setelah) Jam 2");
        }else if (perbandingan < 0) {
            System.out.println("Status                      : Jam 1 lebih awal (sebelum) Jam 2");
        }else {
            System.out.println("Status                      : Jam 1 dan Jam 2 sama persis");
        }

        //operasi aritmatika
        System.out.println("Durasi (J1 ke J2         : " + Jam_2511533002.durasiDetik(j1, j2) + "detik");
```

```java

        Jam_2511533002 JNext = j1.nextSecond();
        System.out.println("Jam 1 detik berikutnya : " + JNext);

        Jam_2511533002 JPrev = j1.prevSecond();
        System.out.println("Jam 1 detik sebelumnya : " + JPrev);

        //operasi penjumlahan jam
        Jam_2511533002 JHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2          : "+ JHasilPlus);

        input.close();
        System.out.println("\nProgram Selesai.");

    }
        private static Jam_2511533002 buatJamDariInput(Scanner sc) {
            int h, m, s;
            while (true) {
                System.out.println("Masukkan Jam (0-23)   : ");
                h = sc.nextInt();
                System.out.println("Masukkan Menit (0-59)   : ");
                m = sc.nextInt();
                System.out.println("Masukkan Detik (0-59)   : ");
                s = sc.nextInt();

                //memanggil method static isvalid dari kelas jam
                if (Jam_2511533002.isValid(h, m, s)) {
                    return new Jam_2511533002(h, m, s);
                }else {
                    System.out.println("[Error] input tidak valid! silahkan ulangi.\n");
                }
            }


}
}
```

- System.out.println("== Program Driver Objek Jam =="); : Mencetak judul program

1. Input Jam

- System.out.println("\n--- Input Jam 1 ---"); : Memberi instruksi bahwa pengguna akan menginput data Jam ke 1.

- Jam_2511533002 j1 = buatJamDariInput(input); : Memanggil fungsi buatJamDariInput.

2. Menampilkan data
    - j1.toString() dan j2.toString() : Menampilkan waktu dalam format hh:mm:ss.
    - j1.toSeconds() dan j2.toSeconds() : Menampilkan total detik dari masing-masing jam yang diinput.
3. Operasi Relasional
    - int perbandingan = j1.compareTo(j2); : Membandingkan Jam 1 dan Jam 2.
4. Operasi Aritmatika
    - Jam_2511533002.durasiDetik(j1, j2) : Menghitung selisih detik antara Jam 1 dan Jam 2.
    - j1.nextSecond() : Membuat objek baru yang waktunya 1 detik setelah Jam 1.
    - j1.prevSecond() : Membuat objek baru yang waktunya 1 detik sebelum Jam 1.
    - j1.plus(j2) — Menjumlahkan Jam 1 dan Jam 2.

    - while (true) { : Membuat perulangan.
    - if (Jam_2511533002.isValid(h, m, s)) : Memanggil method validasi dari kelas Jam.
    - return new Jam_2511533002(h, m, s); : Jika valid, objek jam dibuat, dikirim balik ke main, dan perulangan while otomatis berhenti.
    - else { ... } : Jika user memasukkan angka salah, program mencetak pesan [Error] dan mengulang lagi.

2.2 Output

```
== Program Driver Objek Jam ==

--- Input Jam 1 ---
Masukkan Jam (0-23)   :
10
Masukkan Menit (0-59)   :
38
Masukkan Detik (0-59)   :
42

--- Input Jam 2 ---
Masukkan Jam (0-23)   :
12
Masukkan Menit (0-59)   :
11
Masukkan Detik (0-59)   :
40

--- Hasil Operasi ---
Jam 1 (string)           = 10:38:42
Jam 2 (string)           = 12:11:40
Jam 1 dalam detik        = 36140
Jam 2 dalam detik        = 43311
Status                        : Jam 1 lebih awal (sebelum) Jam 2
Durasi (J1 ke J2         : 7171detik
Jam 1 detik berikutnya : 10:02:21
Jam 1 detik sebelumnya : 10:02:19
Hasil J1 + J2          : 22:124:11

Program Selesai.
```

BAB III
TUGAS

3.1 Uraian Kode Program

1.Class Mobil

```java
package Pekan1_2511533002;

public class Mobil_2511533002 {
        private String nama;
        private int tahun;
        private int cc;
        private long harga;
        private String merk;

        // Constructor (Pembuat Objek)
        public Mobil_2511533002(String nama, int tahun, int cc, long harga, String merk) {
            this.nama = nama;
            this.tahun = tahun;
            this.cc = cc;
            this.harga = harga;
            this.merk = merk;
        }
        public String getNama() { return nama; }
        public int getTahun() { return tahun; }
        public int getCc() { return cc; }
        public long getHarga() { return harga; }
        public String getMerk() { return merk; }
        public void setNama(String nama) { this.nama = nama; }
        public void setTahun(int tahun) { this.tahun = tahun; }
        public void setCc(int cc) { this.cc = cc; }
        public void setHarga(long harga) { this.harga = harga; }
        public void setMerk(String merk) { this.merk = merk; }
        public void infoMobil() {
            System.out.println(merk + " " + nama + " (" + tahun + ") - " + cc + "cc - Rp" + harga);
        }

}
```

2.Class Driver

```java
package Pekan1_2511533002;
import java.util.ArrayList;
public class MobilDriver_2511533002 {
        public static void tambahMobil(ArrayList<Mobil_2511533002> daftar, Mobil_2511533002 mobilBaru) {
            daftar.add(mobilBaru);
            System.out.println("[Sukses] Mobil " + mobilBaru.getNama() + " berhasil ditambahkan.");
        }

        public static void hapusMobil(ArrayList<Mobil_2511533002> daftar, int index) {
            if (index >= 0 && index < daftar.size()) {
                Mobil_2511533002 mobilDihapus = daftar.remove(index);
                System.out.println("[Sukses] Mobil " + mobilDihapus.getNama() + " berhasil dihapus.");
            } else {
                System.out.println("[Gagal] Indeks mobil tidak ditemukan.");
            }
        }

        public static void main(String[] args) {
            ArrayList<Mobil_2511533002> daftarMobil = new ArrayList<>();

            System.out.println("=== PROGRAM DATA SHOWROOM MOBIL ===\n");

            System.out.println("--- Tambah Data ---");
            Mobil_2511533002 mobil1 = new Mobil_2511533002("Innova", 2020, 1300, 150000000, "Toyota");
            Mobil_2511533002 mobil2 = new Mobil_2511533002("Fortuner", 2022, 1200, 160000000, "Honda");

            tambahMobil(daftarMobil, mobil1);
            tambahMobil(daftarMobil, mobil2);

            System.out.println("\n--- Daftar Mobil Saat Ini ---");
            for (int i = 0; i < daftarMobil.size(); i++) {
                System.out.print((i + 1) + ". ");
                daftarMobil.get(i).infoMobil();
            }

            System.out.println("\n--- Uji Coba ---");
            System.out.println("Harga awal " + mobil1.getNama() + " : Rp" + mobil1.getHarga());
            mobil1.setHarga(145000000);
            System.out.println("Harga update " + mobil1.getNama() + " : Rp" + mobil1.getHarga());
            System.out.println("\n--- Proses Hapus Data ---");
            hapusMobil(daftarMobil, 0);

            System.out.println("\n--- Sisa Daftar Mobil ---");
            for (int i = 0; i < daftarMobil.size(); i++) {
                System.out.print((i + 1) + ". ");
                daftarMobil.get(i).infoMobil();
            }
        }
    }
```

3.2 Output

```
=== PROGRAM DATA SHOWROOM MOBIL ===

--- Tambah Data ---
[Sukses] Mobil Innova berhasil ditambahkan.
[Sukses] Mobil Fortuner berhasil ditambahkan.

--- Daftar Mobil Saat Ini ---
1. Toyota Innova (2020) - 1300cc - Rp150000000
2. Honda Fortuner (2022) - 1200cc - Rp160000000

--- Uji Coba ---
Harga awal Innova : Rp150000000
Harga update Innova : Rp145000000

--- Proses Hapus Data ---
[Sukses] Mobil Innova berhasil dihapus.

--- Sisa Daftar Mobil ---
1. Honda Fortuner (2022) - 1200cc - Rp160000000
```

BAB IV
KESMPULAN

Melalui praktikum ini, kita dapat memahami dan mempraktikkan cara pembuatan Abstract Data Type (ADT) pada bahasa pemrograman Java. Secara keseluruhan, tujuan praktikum ini telah tercapai, yaitu memperkenalkan konsep dasar pemrograman berorientasi objek melalui pemisahan kelas objek dan kelas driver. Hal ini memungkinkan mahasiswa untuk beralih dari penulisan kode prosedural biasa ke pemodelan data terstruktur menggunakan selektor dan mutator yang lebih aman, dan modular.

Pemahaman mengenai ADT ini menjadi landasan penting untuk topik pemrograman Java maupun struktur data tingkat lanjut. Dengan demikian, mahasiswa diharapkan mampu mengembangkan logika program dengan rancangan arsitektur data yang terorganisir sesuai kebutuhan.